

Workload Distribution Chart

Advanced Python Engineering Portfolio — Team Scheduling & Role Assignment

Projects: “Stream Forge” (Distributed Systems & Big Data) | “WasmBox” (Security & WebAssembly)

Team size: 6 members per project | Duration: 4 weeks | Schedule: 5 working days per week | Workload: equally distributed across 6 roles

PROJECT 2 – STREAM FORGE

Domain: Distributed Systems & Big Data | Stack: Apache Kafka, Faust/Bytewax, RocksDB, Prometheus, FastAPI, React Flow

- R1: Kafka Infrastructure Lead
- R2: Stream Topology Engineer (Faust/Bytewax)
- R3: Windowing & Aggregation Specialist
- R4: State & Fault-Tolerance Engineer (RocksDB)
- R5: Observability & Reliability Engineer (Prometheus/Chaos)
- R6: React Flow Dashboard Developer

Week 1 – Kafka Foundation & Topology UI Scaffold

Role	Monday	Tuesday	Wednesday	Thursday	Friday
R1: Kafka Infrastructure Lead	Install & configure local Kafka cluster (KRaft/Zookeeper)	Create topics & design partition schema	Write confluent-kafka producer skeleton	Build mock IoT telemetry generator (Truck ID, Temp, Timestamp)	Load-test producer throughput; tune batch/linger settings
R2: Stream Topology Engineer (Faust/Bytewax)	Evaluate Faust vs Bytewax; select framework	Scaffold streaming worker application & config	Define topic schemas (Avro/JSON) & serializers	Build basic consumer connected to Kafka	Integration test consumer against producer
R3: Windowing & Aggregation Specialist	Study tumbling/hopping windowing theory; write design spec	Draft data model for per-truck rolling averages	Prototype in-memory windowing function	Unit test windowing edge cases	Document windowing design for team review
R4: State & Fault-Tolerance Engineer (RocksDB)	Evaluate RocksDB embedding options for Python	Stand up local RocksDB instance	Design key schema for per-truck state	Write read/write wrapper module	Test basic persistence across restarts
R5: Observability & Reliability Engineer (Prometheus/Chaos)	Set up Prometheus + Grafana locally	Define initial metrics taxonomy (events/sec, lag)	Instrument producer with baseline counters	Build broker health-check scripts	Write chaos-testing plan for Week 3
R6: React Flow Dashboard Developer	Scaffold React app with React Flow library	Build topology canvas skeleton	Create placeholder DAG nodes for pipeline stages	Connect scaffold to mock API	Style baseline dashboard theme

Week 2 – Stream Topology & Windowing Logic

Role	Monday	Tuesday	Wednesday	Thursday	Friday
R1: Kafka Infrastructure Lead	Scale producer to simulate 50,000 trucks	Configure partition count & replication factor	Implement producer retry/backoff logic	Benchmark ingestion at target throughput	Document Kafka cluster runbook
R2: Stream Topology Engineer (Faust/Bytewax)	Implement Consume → Filter(Temp>0) → Map graph	Add filtering logic edge cases	Implement map/transform stage	Wire topology output to downstream topic	Code review & refactor topology
R3: Windowing & Aggregation Specialist	Implement tumbling window grouping by timestamp	Implement hopping window variant	Handle late-arriving data logic	Unit tests for windowing correctness	Integrate windowing into streaming topology
R4: State & Fault-Tolerance Engineer (RocksDB)	Integrate state store with per-truck window results	Draft changelog write path	Test state read latency	Handle key eviction/TTL	Sync with Windowing Specialist on state interface
R5: Observability & Reliability Engineer (Prometheus/Chaos)	Add Prometheus counters to stream workers	Expose processing-lag metric	Set up Grafana dashboard panels	Benchmark & record baseline throughput	Prepare mid-project audit metrics report
R6: React Flow Dashboard Developer	Build DAG visualization bound to live topology	Add node status indicators	Connect FastAPI metrics endpoint (stub)	Polish DAG edges/animations	Usability pass on dashboard

Mid-Project Review – Throughput & Windowing Audit

Role	Monday	Tuesday	Wednesday	Thursday	Friday
R1: Kafka Infrastructure Lead	Tune brokers for 100,000 events/sec target	Support throughput audit test run 1	Support throughput audit test run 2	Resolve broker-side bottlenecks	Finalize infrastructure audit report
R2: Stream Topology Engineer (Faust/Bytewax)	Optimize topology for parallel partitions	Profile per-partition throughput	Document topology bottlenecks	Apply topology fixes	Re-test post-optimization throughput
R3: Windowing & Aggregation Specialist	Verify rolling-average math vs test datasets	Validate late-arriving data handling	Run correctness regression suite	Fix identified calculation edge cases	Sign off windowing correctness

Role	Monday	Tuesday	Wednesday	Thursday	Friday
R4: State & Fault-Tolerance Engineer (RocksDB)	Verify state consistency under sustained load	Stress-test RocksDB read/write paths	Prep chaos-testing infrastructure	Validate changelog integrity	Sign off state-layer readiness
R5: Observability & Reliability Engineer (Prometheus/Chaos)	Execute throughput audit tooling	Capture events/sec across partitions	Aggregate audit metrics	Identify performance bottlenecks	Produce mid-project audit report
R6: React Flow Dashboard Developer	Build live audit visualization panel	Wire panel to real-time throughput numbers	Add per-partition breakdown view	Add windowing-correctness status widget	Review panel with team for audit demo

Week 3 – Stateful Recovery & Chaos Testing

Role	Monday	Tuesday	Wednesday	Thursday	Friday
R1: Kafka Infrastructure Lead	Configure Kafka changelog topic	Tune replication for durability	Test broker failover	Validate no data loss on broker restart	Document durability guarantees
R2: Stream Topology Engineer (Faust/Bytewax)	Wire RocksDB state store into topology	Handle state recovery on worker restart	Integration test recovery path	Refine topology fault handling	Code review recovery logic
R3: Windowing & Aggregation Specialist	Validate windowed state survives restarts	Test rolling-average continuity post-recovery	Edge-case testing for partial windows	Run regression test suite	Sign off recovery correctness
R4: State & Fault-Tolerance Engineer (RocksDB)	Implement RocksDB changelog backup to Kafka	Implement restore-from-changelog logic	Test with simulated worker crash	Tune snapshot intervals	Finalize state-recovery module
R5: Observability & Reliability Engineer (Prometheus/Chaos)	Design chaos test (kill Worker #4 mid-calculation)	Verify partition auto-rebalance to Worker #5	Verify state recovered without data loss	Automate chaos-test script	Document chaos-testing results
R6: React Flow Dashboard Developer	Build telemetry dashboard (per-stream metrics, worker health)	Add live partition-rebalancing visualization	Add worker crash/recovery animation	Connect real metrics feed	Polish telemetry UI

Week 4 – Metrics Export, Refine & Polish

Role	Monday	Tuesday	Wednesday	Thursday	Friday
R1: Kafka Infrastructure Lead	Final cluster tuning for production readiness	Security review of Kafka ACLs	Load test at full scale	Fix discovered infra issues	Final infrastructure handoff docs
R2: Stream Topology Engineer (Faust/Bytewax)	Finalize topology code cleanup	Add exactly-once semantics checks	End-to-end integration testing	Bug fixes from integration testing	Final demo preparation
R3: Windowing & Aggregation Specialist	Finalize windowing edge-case documentation	Add configurable window-size feature	Regression testing	Performance tuning	Final correctness sign-off
R4: State & Fault-Tolerance Engineer (RocksDB)	Finalize RocksDB module documentation	Add monitoring hooks for state-store size	Stress test state store at scale	Bug fixes	Final handoff documentation
R5: Observability & Reliability Engineer (Prometheus/Chaos)	Implement Prometheus client metrics across all workers	Connect metrics to Grafana/React dashboard	Validate metric accuracy	Finalize bottleneck alerting rules	Final observability report
R6: React Flow Dashboard Developer	Connect Prometheus metrics to React Flow dashboard	Add bottleneck node highlighting	Final UI polish	Cross-browser testing	Final demo walkthrough

PROJECT 3 – WASMBOX

Domain: Security & WebAssembly (WASM) | Stack: Wasmtime-py, Custom Compiler Toolchain, React, Monaco Editor

R1: WASM Runtime Lead (Wasmtime-py)
R2: Compiler Engineer (Python→WASM)
R3: Security & Sandboxing Specialist
R4: Host Functions / API Bridge Engineer
R5: Developer Portal Frontend Engineer (React/Monaco)
R6: QA, Metrics & Integration Engineer

Week 1 – WASM Foundation & IDE Scaffolding

Role	Monday	Tuesday	Wednesday	Thursday	Friday
R1: WASM Runtime Lead (Wasmtime-py)	Install & configure wasmtime-py runtime	Load a pre-compiled benign .wasm file	Execute wasm module from Python host, capture output	Benchmark execution overhead	Document runtime setup
R2: Compiler Engineer (Python→WASM)	Research Python→WASM toolchains (Pyodide/Rust)	Set up build toolchain	Compile a 'hello world' Python script to WASM	Test compiled output execution	Document compiler pipeline design
R3: Security & Sandboxing Specialist	Define threat model for untrusted plugin execution	Draft security requirements document	Research Wasmtime sandbox capabilities	Prototype basic sandbox configuration	Review threat model with team
R4: Host Functions / API Bridge Engineer	Research host-function bridging patterns in Wasmtime	Design API contract for host calls	Prototype simple host function (logging)	Test host-function call from WASM	Document API bridge design
R5: Developer Portal Frontend Engineer (React/Monaco)	Scaffold React application	Integrate Monaco Editor component	Configure Python syntax highlighting	Build layout (editor + output panel)	Style baseline IDE theme
R6: QA, Metrics & Integration Engineer	Set up test framework & CI skeleton	Write test plan for security audits	Set up plugin-execution logging	Build baseline test harness	Document QA strategy

Week 2 – Compiler & Execution Pipeline

Role	Monday	Tuesday	Wednesday	Thursday	Friday
R1: WASM Runtime Lead (Wasmtime-py)	Integrate dynamic compiled-wasm module loading	Optimize module instantiation time	Handle wasm execution errors gracefully	Test multiple concurrent module loads	Document runtime API
R2: Compiler Engineer (Python→WASM)	Build API endpoint accepting raw Python code	Package code with micro-Python interpreter	Output executable .wasm binary	Test compiler with varied sample scripts	Handle compiler error cases
R3: Security & Sandboxing Specialist	Assist security review of compiler output	Draft sandbox permission defaults (deny fs/network)	Implement baseline capability restrictions	Test restrictions block basic file access	Document security baseline
R4: Host Functions / API Bridge Engineer	Define whitelisted host-function set (v1)	Implement secure calling convention	Add input validation on host calls	Unit test host-function bridge	Document approved API surface
R5: Developer Portal Frontend Engineer (React/Monaco)	Connect 'Run' button to backend compiler API	Display stdout output in UI	Handle loading/error states	Add basic syntax validation feedback	Polish run-flow UX
R6: QA, Metrics & Integration Engineer	Write integration tests for compile→execute flow	Test with sample plugins	Log execution results for QA	Identify edge-case failures	Report findings to team

Mid-Project Review – Security Audit (File System & Network)

Role	Monday	Tuesday	Wednesday	Thursday	Friday
R1: WASM Runtime Lead (Wasmtime-py)	Support security audit test execution	Verify runtime enforces isolation correctly	Investigate any isolation failures	Patch runtime-level gaps	Sign off runtime isolation
R2: Compiler Engineer (Python→WASM)	Fix compiler bugs found during audits	Ensure test plugins compile safely for auditing	Support reproducibility of audit findings	Regression test compiler after fixes	Document compiler audit notes
R3: Security & Sandboxing Specialist	Write malicious plugin attempting to read /etc/passwd	Verify sandbox blocks it & throws permission error	Write plugin attempting external network socket	Verify sandbox intercepts & denies the request	Compile full security audit report
R4: Host Functions / API Bridge Engineer	Verify host-function whitelist cannot be bypassed	Harden input validation against malicious calls	Test boundary conditions on host calls	Patch discovered vulnerabilities	Sign off host-function security
R5: Developer Portal Frontend Engineer (React/Monaco)	Build UI panel to display audit results	Show blocked file-system attempts	Show blocked network attempts	Add pass/fail visual indicators	Review panel with team for audit demo
R6: QA, Metrics & Integration Engineer	Execute full audit test suite	Document pass/fail results	Coordinate fixes with Runtime & Security leads	Re-run audit suite post-fixes	Finalize mid-project audit sign-off

Week 3 – Resource Constraints & Memory Metrics

Role	Monday	Tuesday	Wednesday	Thursday	Friday
R1: WASM Runtime Lead (Wasmtime-py)	Configure Wasmtime memory limits (10MB cap)	Configure instruction/fuel count limits	Implement 50ms execution timeout	Test enforcement with a while-True plugin	Document resource-limit configuration
R2: Compiler Engineer (Python→WASM)	Adjust compiler to respect resource-limit metadata	Test compiled plugins against limits	Handle compiler warnings for oversized code	Regression test compiler changes	Document limits interface
R3: Security & Sandboxing Specialist	Stress-test sandbox with memory-bomb plugin	Verify enforcement doesn't crash host	Test fuel-exhaustion edge cases	Finalize security hardening pass	Sign off resource-limit security
R4: Host Functions / API Bridge Engineer	Extend host functions: authorized DB row write	Add row-level authorization checks	Test unauthorized access attempts are blocked	Finalize host-function security review	Document DB bridge use case
R5: Developer Portal Frontend Engineer (React/Monaco)	Build UI tracking execution time (ms) & RAM used	Connect metrics to backend response payload	Add visual indicators for limit violations	Polish metrics display	Usability testing with sample plugins
R6: QA, Metrics & Integration Engineer	Test timeout enforcement across many plugin samples	Test memory-limit enforcement across samples	Log performance metrics from test runs	Compile Week 3 QA report	Retest previously fixed issues

Week 4 – Host Functions, Refine & Polish

Role	Monday	Tuesday	Wednesday	Thursday	Friday
R1: WASM Runtime Lead (Wasmtime-py)	Finalize runtime configuration for production	Add support for hot-swapping engine files	Load-test many concurrent plugin executions	Fix discovered runtime bugs	Final runtime handoff documentation
R2: Compiler Engineer (Python→WASM)	Finalize compiler pipeline hardening	Add caching for repeated compilations	Final compiler testing	Bug fixes	Final compiler documentation
R3: Security & Sandboxing Specialist	Run final full-system penetration test	Verify all mid-project findings remain fixed	Regression security testing	Sign off final security report	Present security summary to team
R4: Host Functions / API Bridge Engineer	Finalize whitelisted host-function library	Add webhook-trigger host function	Test webhook-triggered plugin execution	Final host-function documentation	Handoff documentation
R5: Developer Portal Frontend Engineer (React/Monaco)	Finalize developer portal UI	Add save-plugin feature	Add simulated webhook trigger button	Final UI polish & accessibility pass	Final demo walkthrough
R6: QA, Metrics & Integration Engineer	Run full regression test suite across app	Verify save/webhook features end-to-end	Final bug bash	Prepare final QA sign-off report	Final demo support